

OE-EC604C — Object Oriented Programming

Semester VI • 3L:0T:0P • 3 Credits

Module 6 Exam Notes: Inheritance

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Fundamentals of Inheritance & Derivation Levels	2
1.1	Base and Derived Classes	2
1.2	Types of Inheritance	2
2	Constructor & Destructor Execution Sequence	2
2.1	Invocation Sequence Rules	2
3	The Diamond Inheritance Ambiguity	3
3.1	The Diamond Problem and Virtual Base Classes	3
4	Containership vs. Inheritance	5

1. Fundamentals of Inheritance & Derivation Levels

Inheritance is the mechanism by which a new class (derived class) is built from an existing class (base class), inheriting its properties and behaviors. This promotes code reusability.

1.1 Base and Derived Classes

A derived class inherits members of a base class. The visibility of inherited members depends on the **inheritance access level** declared:

1. Public Derivation (`class Derived : public Base`):

- Public base members become **public** in the derived class.
- Protected base members become **protected** in the derived class.
- Private base members are **never accessible** directly (can only be accessed via public/protected base functions).

2. Protected Derivation (`class Derived : protected Base`):

- Public and protected base members become **protected** in the derived class.

3. Private Derivation (`class Derived : private Base`):

- Public and protected base members become **private** in the derived class.

1.2 Types of Inheritance

C++ supports five distinct inheritance topologies:

- **Single Inheritance:** One base class, one derived class.
- **Multilevel Inheritance:** A class is derived from another derived class (e.g., A → B → C).
- **Multiple Inheritance:** A derived class has more than one base class (e.g., A and B inherited by C).
- **Hierarchical Inheritance:** Multiple derived classes inherit from a single base class.
- **Hybrid Inheritance:** A combination of two or more inheritance types (e.g., multiple + hierarchical).

2. Constructor & Destructor Execution Sequence

When a derived class object is instantiated, constructors are executed along the inheritance chain to ensure that the entire object is initialized.

2.1 Invocation Sequence Rules

- **Constructors:** Executed from **top to bottom** along the class hierarchy. The base class constructor is executed first, followed by the derived class constructor.
- **Destructors:** Executed in the **exact reverse order** (bottom to top).

- **Initializer Lists:** Parameterized base constructors must be explicitly called using the derived class constructor's member initialization list.

Code Example: Invocation of parameterized base constructors

```
#include <iostream>
class Base {
protected:
    int baseVal;
public:
    Base(int val) : baseVal(val) {
        std::cout << "Base Constructor: " << baseVal << std::endl;
    }
    ~Base() {
        std::cout << "Base Destructor" << std::endl;
    }
};

class Derived : public Base {
private:
    int derivedVal;
public:
    // Call base constructor explicitly in initialization list
    Derived(int bVal, int dVal) : Base(bVal), derivedVal(dVal) {
        std::cout << "Derived Constructor: " << derivedVal << std::endl;
    }
    ~Derived() {
        std::cout << "Derived Destructor" << std::endl;
    }
};
```

3. The Diamond Inheritance Ambiguity

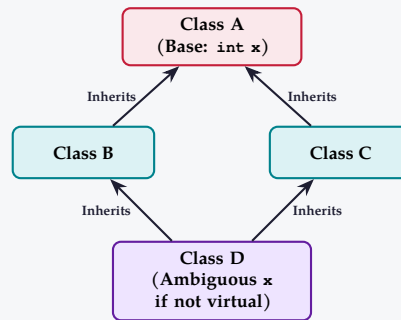
Multipath inheritance can result in a duplicate copy conflict known as the **Diamond Problem**.

3.1 The Diamond Problem and Virtual Base Classes

Consider class **D** inheriting from classes **B** and **C**, which both inherit from class **A**. If **A** contains a member **x**, **D** receives two separate copies of **x** (one via **B** and one via **C**). Any reference to **x** inside **D** is flagged as **ambiguous** by the compiler.

Virtual Base Class: By declaring the base class **A** as `virtual` during derivation in **B** and **C**, the compiler ensures that only one shared instance of class **A** is created in memory, resolving all inheritance path ambiguities.

Diamond Inheritance Path Topologies



Without `virtual`: D has duplicate copies of A's members.
 With `virtual`: D shares a single, resolved instance of A.

Code Example: Resolving Diamond Problem using Virtual Base Class

```

#include <iostream>
class A {
public:
    int x;
};

// Declare B and C with virtual public inheritance
class B : virtual public A {
public:
    int y;
};

class C : virtual public A {
public:
    int z;
};

class D : public B, public C {
public:
    int sum;
    void show() {
        x = 10; // Ambiguous without virtual keyword! Now resolved.
        y = 20;
        z = 30;
        sum = x + y + z;
        std::cout << "Sum = " << sum << std::endl;
    }
};
  
```

4. Containership vs. Inheritance

To design clean systems, software developers must choose the correct structural relationship between classes:

- **Inheritance (is-a relationship):** A derived class inherits properties of the base class. E.g., a Car **is a** Vehicle.
- **Containership (has-a relationship):** An object of one class is embedded as a data member inside another class. E.g., a Car **has an** Engine.

Quick Revision — Module VI

Inheritance | OE-EC604C

Access Control Table under Derivation

Base Member	Public Derivation	Protected Derivation	Private Derivation
Private	Not Accessible	Not Accessible	Not Accessible
Protected	Protected in Derived	Protected in Derived	Private in Derived
Public	Public in Derived	Protected in Derived	Private in Derived

One-Line Definitions

- **Inheritance:** The mechanism of deriving a new class from an existing base class to reuse code.
- **Base Class:** The existing class from which features are inherited.
- **Derived Class:** The new class that inherits features from one or more base classes.
- **Diamond Problem:** An ambiguity issue where a derived class inherits duplicate members of a grandparent class through two distinct paths.
- **Virtual Base Class:** A base class declared virtual to ensure only one instance is shared in multipath inheritance.
- **Containership:** Embedding an object of one class as a member variable inside another class (Composition).
- **Initialization List (Base):** The derived constructor syntax used to pass arguments to base class constructors.
- **Is-A Relationship:** Logical relationship modeled by class inheritance (e.g., Dog is-a Mammal).
- **Has-A Relationship:** Logical relationship modeled by class containership (e.g., Dog has-a Tail).

Frequently Asked / Viva Questions

Q1. Why are private base class members not accessible in derived classes?

This maintains the privacy boundaries of class design. If derived classes could access private members directly, encapsulation could be easily broken.

Q2. Explain why virtual inheritance is necessary to solve the Diamond Problem.

Virtual inheritance directs the compiler to build only one shared instance of the grandparent class, resolving lookup ambiguity for its members.

Q3. What is the execution order of constructors in multiple inheritance?

Base constructors are executed in the order they are listed in the inheritance

Frequently Asked / Viva Questions (contd.)

declaration, NOT the order they appear in the initializer list. E.g., `class D : public B, public C` calls B's constructor first, then C's.

Q4. Which class constructor is executed first in multilevel inheritance?

The absolute root base class constructor is executed first, followed by intermediate bases, ending at the leaf derived class.

Q5. Does a derived class inherit the constructor of the base class?

No. Constructors are class-specific and are not inherited. A derived class must define its own constructors or use the default constructor.

Q6. Can we declare a destructor virtual? Why?

Yes, virtual destructors are mandatory when deleting derived objects via base pointers to ensure derived resource cleanup.

Q7. What is the difference between multiple inheritance and hybrid inheritance?

Multiple inheritance involves deriving from 2+ distinct base classes. Hybrid inheritance combines multiple topologic models (like multiple and multilevel).

Q8. How does containership differ from private inheritance?

Containership embeds objects as members, restricting them to public APIs. Private inheritance makes base members private, permitting code access but hiding public APIs from external callers.

Q9. What happens if base and derived classes declare members with the same name?

The derived member hides the base member. The base member can still be accessed using the scope resolution operator: `BaseClass::memberName`.

Q10. Can a class inherit from itself in C++?

No, circular or self-inheritance (e.g., `class A : public A`) is syntactically invalid and causes compile errors.

Q11. Why does virtual inheritance require base class constructors to be called by the most derived class?

Since the base class instance is shared, intermediate derived classes cannot initialize it independently. The most derived class constructor executes its initialization.

Q12. How does the visibility level of inheritance affect derived class inheritance hierarchies?

Private inheritance prevents downstream subclasses from accessing grandparent class members, whereas public/protected inheritances allow subclass access.

Comparison: Multiple Inheritance vs. Hybrid Inheritance

Feature	Multiple Inheritance	Hybrid Inheritance
Structure	A single subclass derives directly from two or more parent classes.	Combines two or more basic inheritance shapes (e.g., hierarchic, multilevel, multiple).
Diamond Risk	None, unless parent classes share a common grandparent class.	High. Multipath inheritance frequently creates diamond path ambiguities.
Complexity	Moderate. Simple to resolve using qualifiers.	High. Requires virtual inheritance declaration to resolve duplicate members.
Design Goal	Merging features from multiple independent source interfaces.	Modeling complex real-world classification hierarchies.